MILP perks

fast
based on LP + enumeration
+ advanced features

declarative
create the model,
apply a solver

generic & specific
algorithms

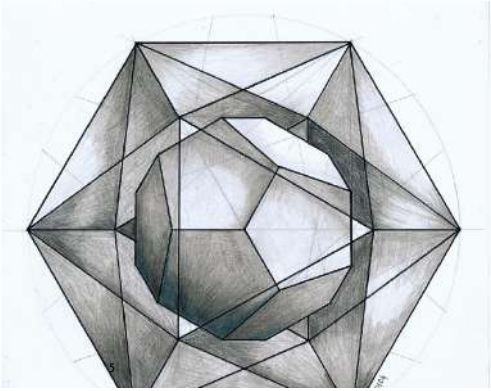
optimality
primal-dual bounds

expressive
logic, nonlinear, discrete
many decision problems

flexible
change the model,
not the solver

1. **how to model ?**
   techniques & applications
2. **how difficult ?**
   complexity & distance to LP
3. **how to solve ?**
   main techniques & modern solvers
   decomposition methods

1 how to model ?



Mixed Integer Linear Program

$$\min cx$$
$$Ax \geq b$$
$$x \in \mathbb{Z}^p \times \mathbb{R}^{n-p}$$

$c \in \mathbb{R}^n, A \in \mathbb{R}^{m \times n}, b \in \mathbb{R}^m$

objective	cx
linear constraints	$Ax \geq b$
integrality constraints	$x_j \in \mathbb{Z}$
right hand side	b
cost vector	c
solution space	$\mathbb{R}^n$
feasible set	$\{x \in \mathbb{Z}^p \times \mathbb{R}^{n-p} \mid Ax \geq b\}$

~~true~~ 1 or ~~false~~ 0

- select item j $\quad x_j = 1, \quad x_j \in \{0,1\}$
- associate item j to resource i $\quad x_{ij} = 1, \quad x_{ij} \in \{0,1\}$
- variable $y \geq 0$ greater than constant a ? $\quad y \geq ax, x \in \{0,1\}$
- select at most n items $\quad x_1, \ldots, x_n \in \{0,1\}$

Integer Knapsack Problem

$$\max \sum_{j=1}^{n} c_j x_j$$
$$\text{s.t.} \sum_{j=1}^{n} w_j x_j \leq K$$
$$x_j \in \{0,1\} \qquad j = 1..n$$

Input n items, value c_j and weight w_j for each item j, capacity K.
Output a maximum value subset of items whose total weight does not exceed K.

x_j is item j packed ?

logic with binaries

x,y binary variables; f continuous variable; a, k, n constants

- either x or y $\quad x + y = 1$
- if x then y $\quad y \geq x$
- if x then f ≤ a $\quad f \leq ax + M(1-x)$
- at most 1 out of n
- at least k out of n

"big M"
big enough but
keep it tight !

logic with binaries

x,y binary variables; f continuous variable; a, k, n constants

- either x or y $\quad x + y = 1$
- if x then y $\quad y \geq x$
- if x then f ≤ a $\quad f \leq ax + M(1-x)$
- at most 1 out of n $\quad x_1 + \cdots + x_n \leq 1$
- at least k out of n $\quad x_1 + \cdots + x_n \geq k$

$$\min \sum_{j=1}^{n} c_j x_j + \sum_{j=1}^{n}\sum_{i=1}^{m} d_{ij} y_{ij}$$

$$\text{s.t.} \sum_{j=1}^{n} y_{ij} = 1 \qquad i = 1..m$$

$$y_{ij} \leq x_j \qquad j = 1..n,\ i = 1..m$$

$$x_j \in \{0,1\} \qquad j = 1..n$$

$$y_{ij} \in \{0,1\} \qquad j = 1..n,\ i = 1..m$$

Uncapacitated Facility Location Problem

Input n facility locations, m customers, cost c_j to open facility j, cost d_{ij} to serve customer i from facility j
Output a mimimum (opening and service) cost assignment of customers to facilities.

x_j is location j open ? y_{ij} is customer i served from j ?

$$\min \sum_{i=1}^{n}\sum_{j=1}^{n} d_{ij} y_{ij}$$

$$\text{s.t.} \sum_{j=1}^{n} y_{ij} = 1 \qquad i = 1..n$$

$$y_{ij} \leq x_j \qquad i, j = 1..n$$

$$\sum_{j=1}^{n} x_j = k$$

$$y_{ij} \in \{0,1\}, x_j \in \{0,1\} \qquad i, j = 1..n$$

K-median clustering

Input n data points, distance d_{ij} between each two points i,j, number k of clusters.
Output k centers minimizing the sum of distances between each point and its nearest center.

x_j is j a center ? y_{ij} is j the nearest center of i ?

Input n data points $m_j \in \mathbb{R}^p$, a number K of clusters. Euclidean distance.

K-median clustering

Output define K points as centers so as to minimize the sum of the distances between each point and its nearest center.

K-mean clustering

Output partition the points into K sets so as to minimize the sum of the distances between each point and the mean of points in its cluster.

K-mean clustering

cannot precompute the distance to the centers anymore: modeled with nonlinear constraints

x_{jk} is j assigned to cluster k ?
y_k coordinates of the center of k ?
d_{jk} distance from j to the center of k ?

$$min \sum_{k=1}^{K} \sum_{j=1}^{n} x_{jk} d_{jk}$$

$$s.t. \; d_{jk} = \sum_{i=1}^{p} (m_j^i - y_k^i)^2 \quad \forall j,k$$

$$\sum_{k=1}^{K} x_{jk} = 1 \quad \forall j$$

$$x_{jk} \in \{0,1\}, y_k^i \in \mathbb{R}, d_{jk} \geq 0$$

non convex

K-mean clustering

x_{jk} is j assigned to cluster k ?
y_k coordinates of the center of k ?
d_{jk} distance from j to the center of its cluster k ?

$$min \sum_{k=1}^{K} \sum_{j=1}^{n} d_{jk}$$

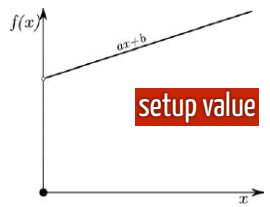
$$s.t. \; d_{jk} \geq \sum_{i=1}^{p} (m_j^i - y_k^i)^2 - \bar{d}_{jk}(1 - x_{jk}) \quad \forall j,k$$

$$\sum_{k=1}^{K} x_{jk} = 1 \quad \forall j$$

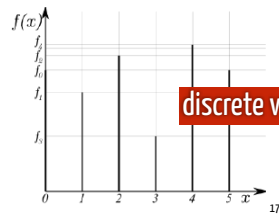
$$x_{jk} \in \{0,1\}, y_k^i \in \mathbb{R}, d_{jk} \geq 0$$

convexify the nonlinear constraints using big-M (optimization is still nonconvex because of integrality)

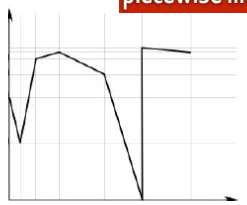
non-linear functions



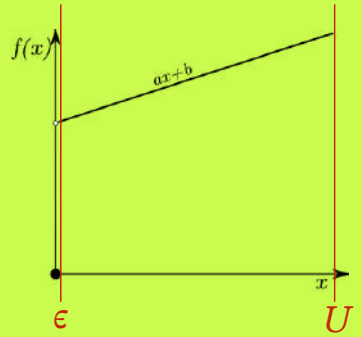
setup value



discrete values



piecewise linear

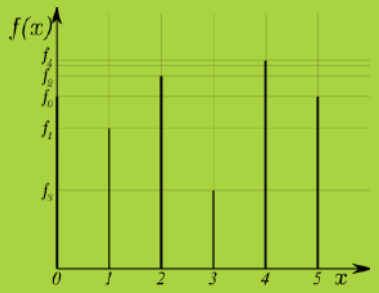


setup value

$$f(x) = ax + b\delta$$
$$\epsilon\delta \le x \le U\delta$$
$$\delta \in \{0,1\}$$

δ is x positive ?

Special Ordered Set of type 1:
ordered set of variables, all zero except at most one



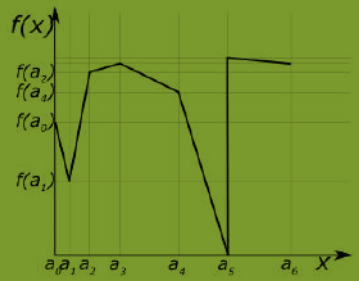
discrete values

$$f(x) = \sum_i \delta_i f_i$$
$$\sum_i i\delta_i = x$$
$$\sum_i \delta_i \geq 1$$
$$\delta_i \in \{0,1\} \; i = 0..n$$

SOS1(δ)

δ_i is $x=i$ (and $f(x)=f_i$) ?

Special Ordered Set of type 2:
ordered set of variables, all zero except at most two consecutive



piecewise linear

$$f(x) = \sum_i \lambda_i f(a_i)$$
$$\sum_i a_i\lambda_i = x$$
$$\sum_i \lambda_i = 1$$
$$\lambda_i \in [0,1] \; i = 0..n$$

SOS2(λ)

λ_i is $x=a_i$? (then $\lambda_i a_i + \lambda_{i+1} a_{i+1}$ in $[a_i, a_{i+1}]$ if $\lambda_i + \lambda_{i+1} = 1$)

$$x_i = 5$$

to order i is the 5th item

to count 5 items are selected

to measure time task i starts at time 5

to measure space item i is located on floor 5

$$\simeq \delta_{i5} = 1$$

Binary Integer Linear Program (BIP) $\{0,1\}^n$

Integer Linear Program (IP) $\mathbb{Z}^n$

Mixed Integer Linear Program (MIP) $\mathbb{Z}^n \cup \mathbb{Q}^n$

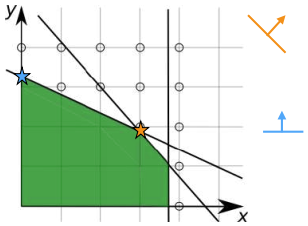
1 how to model ?

2 how difficult ?

3 how to solve ?

~~Linear~~ Programming cheat sheet

LP is easy



- MILP without integrality = LP relaxation
- LP feasible set = polyhedron
- convex optimization
- if LP is feasible and bounded, at least one vertex is optimal
- primal simplex algorithm: visit adjacent vertices as cost decreases
- strong duality: $\min\{cx \mid Ax \geq b, x \geq 0\} = \max\{ub \mid uA \leq c, u \geq 0\}$
- interior point method runs in polynomial time (simplex can be better in practice)

Market Split Problem

$$\min \sum_{j=1}^{m} s_j^+ + s_j^-$$

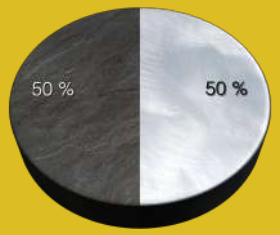
$$\text{s.t.} \sum_{i=1}^{n} a_{ij} x_i + s_j^+ - s_j^- = \frac{d_j}{2} \qquad j = 1..m$$

$$x_i \in \{0,1\} \qquad i = 1..n$$

$$s_j^+ \geq 0, s_j^- \geq 0 \qquad j = 1..m$$

Input 1 company, 2 divisions, m products with availabilities d_j, n retailers with demands a_{ij} in each product j.
Output an assignment of the retailers to the divisions approaching a 50/50 production split.

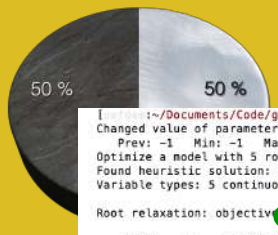
x_i is retailer i assigned to division 1 ?
s_j gap to the 50% split goal for product j



MIPLIB markshare_5_0

Input 5 products, 40 retailers
Output . . . . . .
. . . . . (hold the line please) . . . .

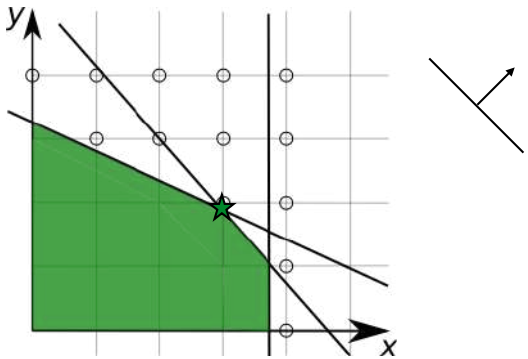
Int Opt = 1
Solution time = 20 minutes
Proof time = > 1 hour



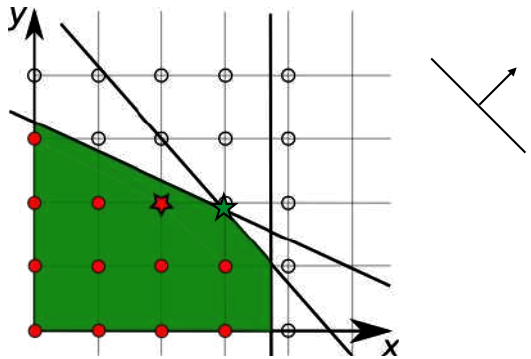
MIPLIB markshare_5_0

```
[redacted]:~/Documents/Code/gurobi]$ gurobi.sh mymip.py markshare_5_0.mps.gz
Changed value of parameter Presolve to 0
   Prev: -1   Min: -1   Max: 2   Default: -1
Optimize a model with 5 rows, 45 columns and 203 nonzeros
Found heuristic solution: objective 5335
Variable types: 5 continuous, 40 integer (40 binary)

Root relaxation: objective 0.00000[illegible]0e+00, 15 iterations, 0.00 seconds

    Nodes    |    Current Node    |     Objective Bounds      |     Work
 Expl Unexpl |  Obj  Depth IntInf | Incumbent    BestBd   Gap | It/Node Time

     0     0    0.00000    0    5 5335.00000    0.00000   100%     -    0s

*62706364 28044              38       1.0000000    0.00000   100%   2.1 1241s

Explored 233848403 nodes (460515864 simplex iterations) in 3883.56 seconds
Thread count was 4 (of 4 available processors)

Optimal solution found (tolerance 1.00e-04)
Best objective 1.000000000000e+00, best bound 1.000000000000e+00, gap 0.0%
Optimal objective 1
```

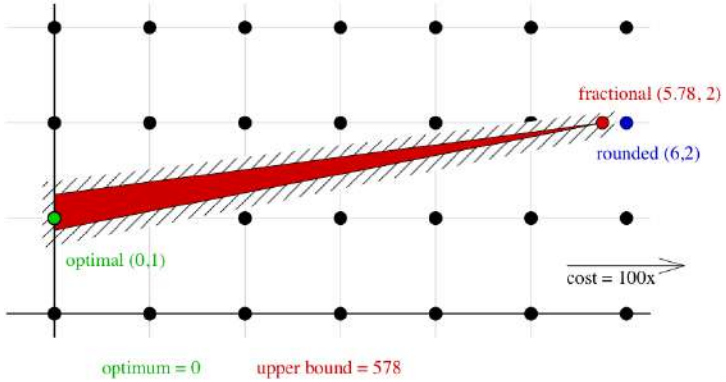

ILP ≠ LP relaxation



ILP ≠ LP relaxation



ILP ≠ round LP relaxation



general ILP is NP-hard

small problems are easy
some specific problems are easy

1||Cmax Scheduling Problem

$$
\begin{aligned}
\min\ & s_{n+1} \quad = p_1 + \ldots + p_n \\
\text{s.t. } & s_{n+1} \geq s_j + p_j && j = 1..n \\
& s_j - s_i \geq M x_{ij} + (p_i - M) && i, j = 1..n \\
& x_{ij} + x_{ji} = 1 && i, j = 1..n; i < j \\
& s_j \in \cancel{\mathbb{Z}_+} \geq 0 && j = 1..n+1 \\
& x_{ij} \in \{0, 1\} && i, j = 1..n
\end{aligned}
$$

Input n tasks, duration p_i for each task i, 1 machine
Output a minimal makespan schedule of the tasks on the machine without overlap

Capacitated Transhipment Problem

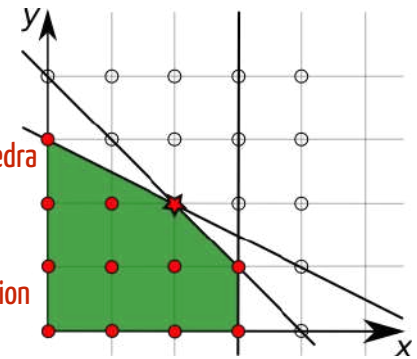
$$
\begin{aligned}
\min\ & \sum_{(i,j) \in A} c_{ij} x_{ij} \\
\text{s.t. } & \sum_{j \in \delta^+(i)} x_{ij} - \sum_{j \in \delta^-(i)} x_{ij} = b_i && i \in V \\
& x_{ij} \leq h_{ij} && (i,j) \in A \\
& x_{ij} \in \cancel{\mathbb{Z}_+} \geq 0 && (i,j) \in A
\end{aligned}
$$

Input digraph (V,A), demand or supply b_i at each node i, capacity h_{ij} and unit flow cost c_{ij} for each arc (i,j)
Output a mimimum cost integer flow to satisfy the demand

x_{ij} flow on arc (i,j)

LP = ILP sometimes

integral polyhedra
=
convex hull
=
ideal formulation



totally unimodular matrix
(theory)

$$(P) = \max\{\ cx \mid Ax \leq b, x \in \mathbb{Z}^n_+\ \}$$

- basic feasible solutions of the LP relaxation $(\bar{P})$ take the form: $\bar{x} = (\bar{x}_B, \bar{x}_N) = (B^{-1}b, 0)$ where B is a square submatrix of (A, I_m)
- Cramer's rule: $B^{-1} = B^*/det(B)$ where B^* is the adjoint matrix (made of products of terms of B)
- Proposition: if (P) has integral data (A, b) and if $det(B) = \pm 1$ then $\bar{x}$ is integral

Definition

A matrix A is totally unimodular (TU) if every square submatrix has determinant $+1$, -1 or 0.

Proposition

If A is TU and b is integral then any optimal solution of $(\bar{P})$ is integral.

totally unimodular matrix
(practice)

How to recognize TU ?

Sufficient condition

A matrix A is TU if

- all the coefficients are $+1$, -1 or 0
- each column contains at most 2 non-zero coefficient
- there exists a partition (M_1, M_2) of the set M of rows such that each column j containing two non zero coefficients satisfies $\sum_{i \in M_1} a_{ij} - \sum_{i \in M_2} a_{ij} = 0$.

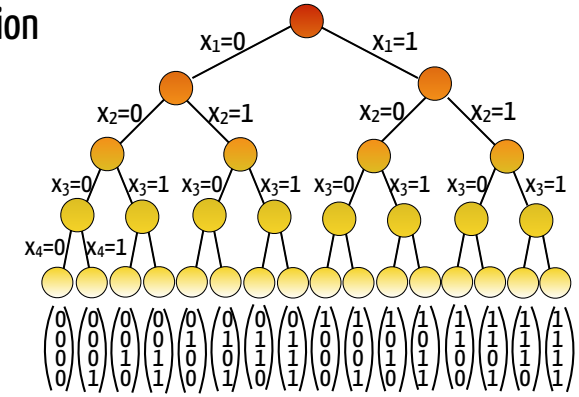
Proposition

A is TU $\iff$ A^t is TU $\iff$ (A, I_m) is TU
where A^t is the transpose matrix, I_m the identitiy matrix

Interlude

Show that the Transhipment ILP is ideal
Show that the Scheduling ILP is NOT ideal

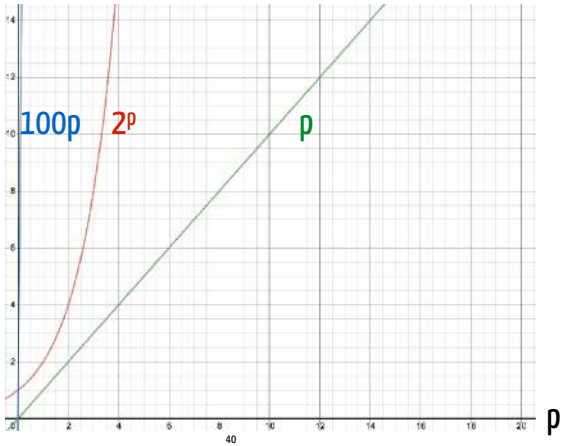
Complete enumeration



MILP with p binaries

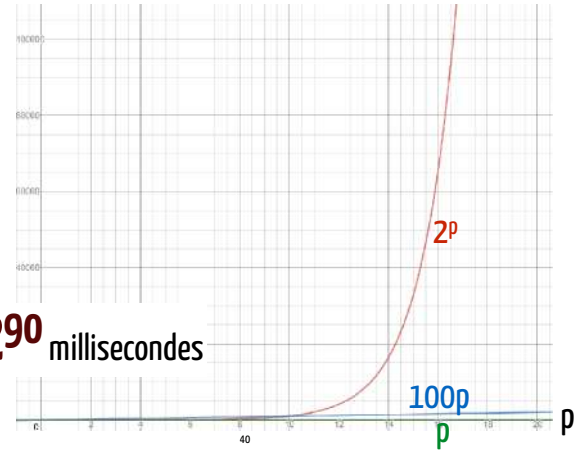
$$\min\{cx \mid Ax \geq b, x \in \{0,1\}^p \times \mathbb{R}^{n-p}\}$$ = 2^p LPs to solve

Combinatorial explosion



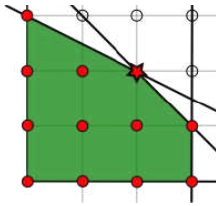
Combinatorial explosion

âge de l'univers ≈ 2^{90} millisecondes

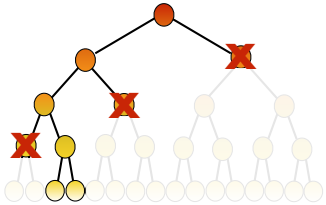


Two options

1 compute
an ideal formulation



2 evaluate partial solutions
progressively

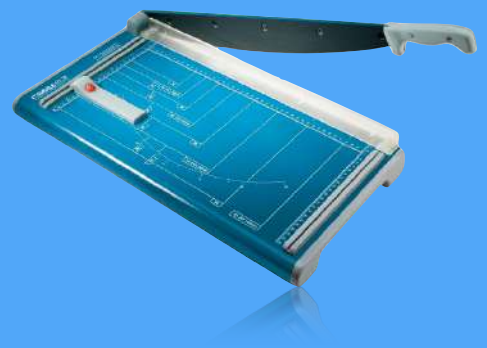


1 Cut Generation compute an ideal formulation

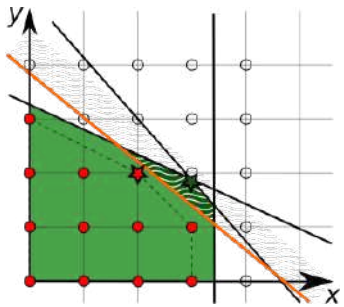
2 Branch&Bound evaluate partial solutions progressively

3 modern Branch&Cut mix up+presolve+heuristics

4 decomposition methods (Branch&Price, Lagrangian relaxation, Benders)



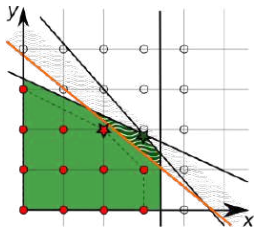
Cutting Plane Algorithm



Cut valid inequality that separates a relaxed LP solution

Farkas Lemma cuts are linear combinations of constraints

cutting plane algorithm




```
1. solve the LP relaxation of (P), get x̄
2. if x̄ is integral STOP:
   feasible then optimal for (P)
3. find cuts C for (P,x̄) from template T
4. add constraints C to (P) then 1.
```

separation subproblem

templates

general-purpose
mixed integer rounding, split, Chvátal-Gomory

structure-based
clique, cover, flow cover, zero half

problem-specific
subtour elimination (TSP), odd-set (matching)

ex 1 Chvátal-Gomory cuts

$(P) : \max\{cx \mid Ax \leq b, x \in \mathbb{Z}_+\}$

For any $u \in \mathbb{R}^m_+$ the following inequalities are valid:

1. surrogate: $\sum_j \sum_i u_i\, a_{ij}\, x_j \leq \sum_i u_i\, b_i$ $\quad (u \geq 0)$
2. round off: $\sum_j \lfloor \sum_i u_i\, a_{ij} \rfloor x_j \leq \sum_i u_i\, b_i$ $\quad (x \geq 0)$
3. Chvátal-Gomory: $\sum_j \lfloor \sum_i u_i\, a_{ij} \rfloor x_j \leq \lfloor \sum_i u_i\, b_i \rfloor$ $\quad (\lfloor uA \rfloor x \in \mathbb{Z})$

variants in the choice of u, ex: Gomory or MIR cuts

ex 2 Cover cuts

$$S = \{y \in \{0,1\}^7 | 11y_1 + 6y_2 + 6y_3 + 5y_4 + 5y_5 + 4y_6 + y_7 \leq 19\}$$

- (y_3, y_4, y_5, y_6) is a minimal cover for $11y_1 + 6y_2 + 6y_3 + 5y_4 + 5y_5 + 4y_6 + y_7 \leq 19$ as $6+5+5+4 > 19$ then $y_3 + y_4 + y_5 + y_6 \leq 3$ is a cover inequality
- we can derive a stronger valid inequality $y_1 + y_2 + y_3 + y_4 + y_5 + y_6 \leq 3$ by noting that y_1, y_2 has greater coefficients than any variable in the cover
- note furthermore that (y_1, y_i, y_j) is a cover $\forall i \neq j \in \{2,3,4,5,6\}$ then $2y_1 + y_2 + y_3 + y_4 + y_5 + y_6 \leq 3$ is also valid

lifting

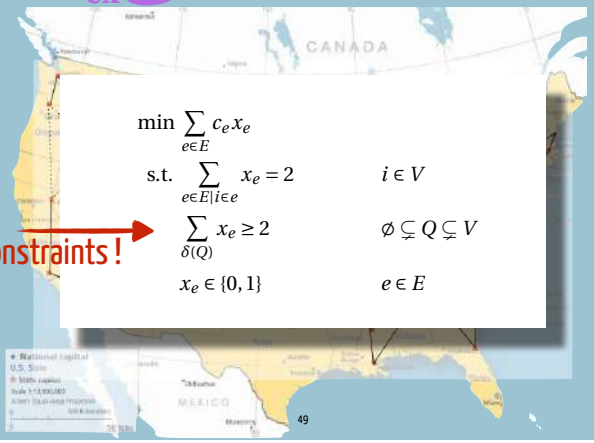
separation: solve knapsack $\min\{\sum_j (1-\bar{y}_j)x_j \mid \sum a_j x_j \geq b + \epsilon, x \in \{0,1\}^n\}$

get coefficients x^* of the cover inequality $\sum x_j^* y_j \leq \sum x_j^* - 1$

if $\sum (1-\bar{y}_j)x_j^* < 1$ then it cuts off the current LP solution $\bar{y}$

ex 3 Subtour for TSP

ex 3 Subtour for TSP



$$\min \sum_{e \in E} c_e x_e$$

$$\text{s.t.} \sum_{e \in E \mid i \in e} x_e = 2 \qquad i \in V$$

$$\sum_{\delta(Q)} x_e \geq 2 \qquad \emptyset \subsetneq Q \subsetneq V$$

$$x_e \in \{0,1\} \qquad e \in E$$

2^n constraints !

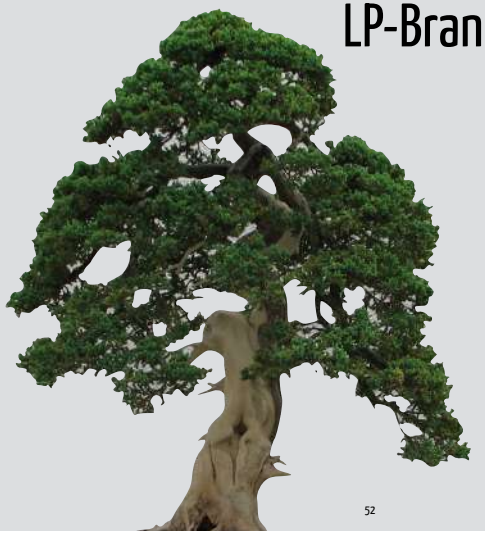
ex 3 Subtour for TSP

separation: solve min s-t cut in $(V, \overrightarrow{E}, \bar{x})$ for some fixed s and each $t \in V \setminus \{s\}$ to find a cutset $\delta(Q)$ of capacity < 2 or prove none exists

limits depending on the templates

- the algorithm may stop prematurely
- the algorithm may not converge
- the algorithm may converge slowly
- the separation procedure may be NP-hard
- the LP relaxation grows
- the LP relaxation structure changes

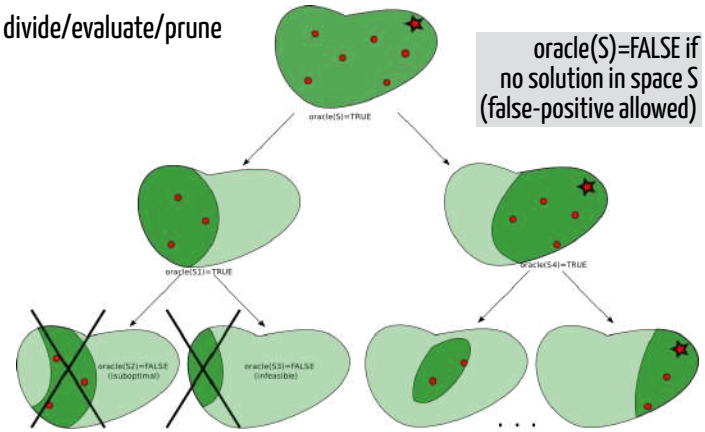
LP-Branch and Bound



Search tree

divide/evaluate/prune

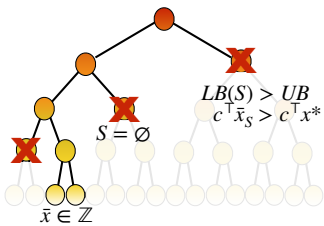
oracle(S)=FALSE if
no solution in space S
(false-positive allowed)

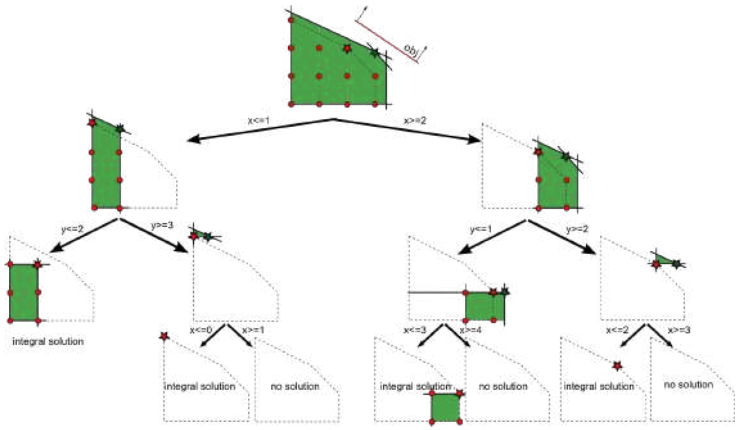


LP-based branch and bound

1. evaluate by solving the LP relaxation and compare bounds
2. divide the variable domains

```
oracle(S) = FALSE if either:
 - the LP relaxation is unfeasible on S
 - the relaxed LP solution x̄ is not
   better than the best integer solution
   found so far x*
 - x̄ is integer (then update x*)
```





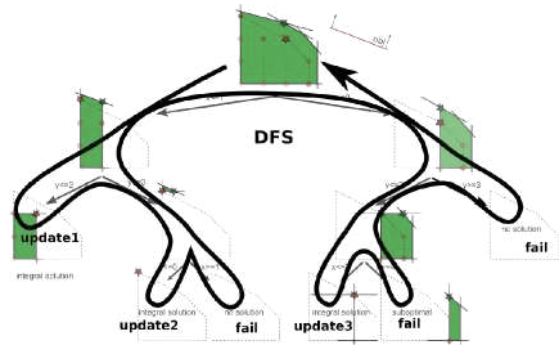
branching

node selection
which order to visit nodes ?

variable selection
how to separate nodes ?

constraint branching
versus variable branching

node selection

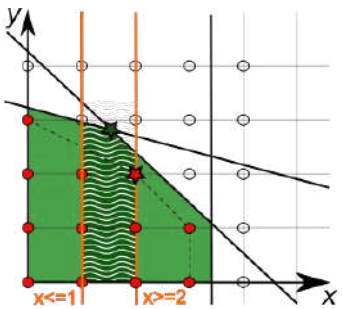


Best Bound First Search explore less nodes, manages larger trees

Depth First Search sensible to bad decisions at or near the root

DFS (up to n solutions) + BFS (to prove optimality)

variable selection



most fractional easy to implement but not better than random

strong branching best improvement among all candidates (impractical)

pseudocost branching record previous branching success for each var (inaccurate at root)

reliability branching pseudocosts initialised with strong branching

constraint branching

example: GUB dichotomy

- if (P) contains a GUB constraint $\sum_C x_i = 1, x \in \{0,1\}^n$
- choose $C' \subseteq C$ s.t. $0 < \sum_{C'} \bar{x}_i < 1$
- create two child nodes by setting either $\sum_{C'} x_i = 0$ or $\sum_{C'} x_i = 1$

- enforced by fixing the variable values
- leads to more balanced search trees

SOS1 branching in a facility location problem

choose a warehouse depending on its size/cost:

$$\begin{aligned} \text{COST} &= 100x_1 + 180x_2 + 320x_3 + 450x_4 + 600x_5 \\ \text{SIZE} &= 10x_1 + 20x_2 + 40x_3 + 60x_4 + 80x_5 \\ \text{(SOS1)} &: x_1 + x_2 + x_3 + x_4 + x_5 = 1 \end{aligned}$$

- let $\bar{x}_1 = 0.35$ and $\bar{x}_5 = 0.65$ in the LP solution then SIZE$= 55.5$
- choose $C' = \{1,2,3\}$ in order to model SIZE≤ 40 or SIZE≥ 60

modern solvers

60

LP solver

preprocessing

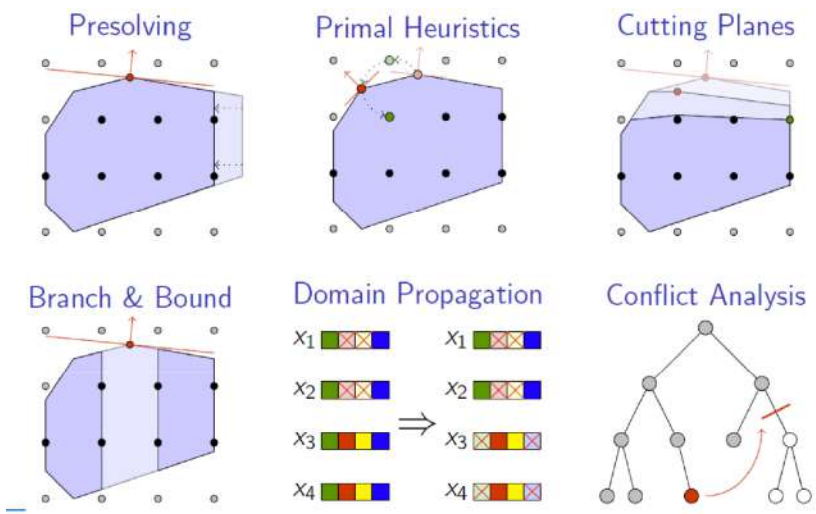
strategies

Branch & Cut

heuristics

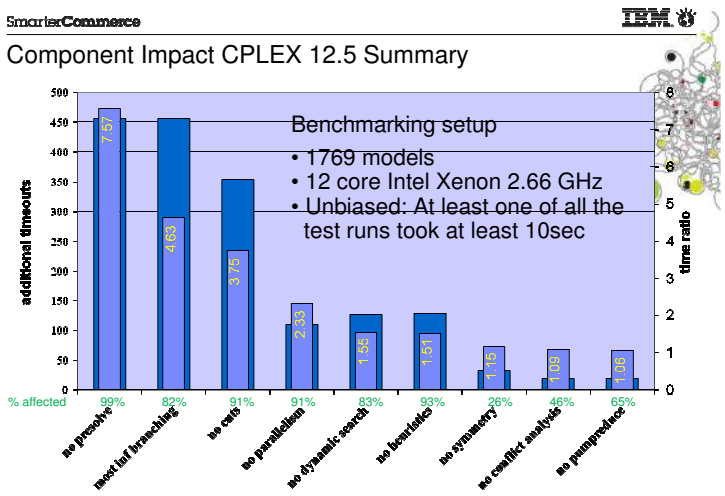
parallelism

61



62

Slide from Martin Grötschel 2015

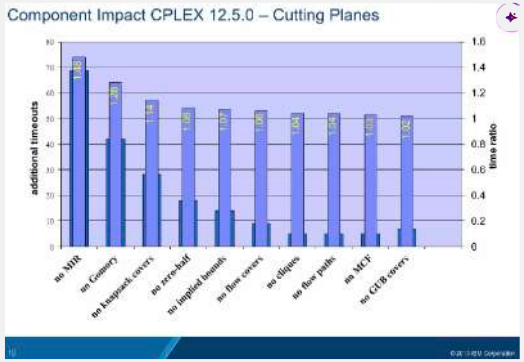


12

63

© 2013 IBM Corporation

GUROBI 12



BQPCuts	BQP cut generation
Cuts	Global cut generation control
CliqueCuts	Clique cut generation
CoverCuts	Cover cut generation
CutAggPasses	Constraint aggregation passes performed during cut generation
CutPasses	Root cutting plane pass limit
DualImpliedCuts	Dual implied bound cut generation
FlowCoverCuts	Flow cover cut generation
FlowPathCuts	Flow path cut generation
GomoryPasses	Root Gomory cut pass limit
GUBCoverCuts	GUB cover cut generation
ImpliedCuts	Implied bound cut generation
InfProofCuts	Infeasibility proof cut generation
LiftProjectCuts	Lift-and-project cut generation
MIPSepCuts	MIP separation cut generation
MIRCuts	MIR cut generation
MixingCuts	Mixing cut generation
ModKCuts	Mod-k cut generation
NetworkCuts	Network cut generation
ProjImpliedCuts	Projected implied bound cut generation
PSDCuts	PSD cut generation
RelaxLiftCuts	Relax-and-lift cut generation
RLTCuts	RLT cut generation
StrongCGCuts	Strong-CG cut generation
SubMIPCuts	Sub-MIP cut generation
ZeroHalfCuts	Zero-half cut generation

Preprocessing

reduce size

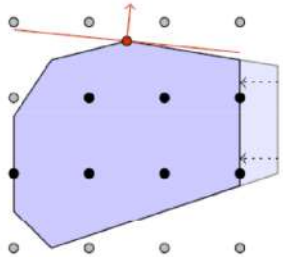
remove redundancies	$x+y\le 3$, binaries
substitute variables	$x+y-z=0$
fix variables by duality	$c_j\ge 0,\ A_j\ge 0 \Rightarrow x=x_{min}$
fix variables by probing	$x=1$ infeas $\Rightarrow x=0$

strengthen LP relaxation

adjust bounds	$2x+y\le 1$, binaries $\Rightarrow x=0$
lift coefficients	$2x-y\le 1$, binaries $\Rightarrow x-y\le 1$

identify/exploit properties

detect implied integer	$3x+y=7$, x int $\Rightarrow$ y int
build the conflict graph	
detect disconnected components	
remove symmetries	

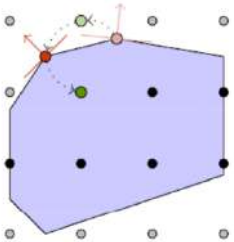


MIPLIB markshare_5_0

```
[sofdem:~/Documents/Code/gurobi]$ gurobi.sh mymip.py markshare_5_0.mps.gz
Changed value of parameter Presolve to 0
   Prev: -1  Min: -1  Max: 2  Default: -1
Optimize a model with 5 rows, 45 columns and 203 nonzeros
Found heuristic solution: objective 5335
Variable types: 5 continuous, 40 integer (40 binary)

Root relaxation: objective 0.000000e+00, 15 iterations, 0.00 seconds

    Nodes    |    Current Node    |     Objective Bounds      |     Work
 Expl Unexpl |  Obj  Depth IntInf | Incumbent    BestBd   Gap | It/Node Time

     0     0    0.00000    0    5 5335.00000    0.00000   100%     -    0s

*62706364 28044              38       1.0000000    0.00000   100%   2.1 1241s

Explored 233848403 nodes (460515864 simplex iterations) in 3883.5 seconds
Thread count was 4 (of 4 available processors)

Optimal solution found (tolerance 1.00e-04)
Best objective 1.000000000000e+00, best bound 1.000000000000e+00, gap 0.0%
Optimal objective: 1
```



```
[sofdem:~/Documents/Code/gurobi]$ gurobi.sh mymip.py markshare_5_0.mps.gz
Optimize a model with 5 rows, 45 columns and 203 nonzeros
Found heuristic solution: objective 5335
Presolve time: 0.00s
Presolved: 5 rows, 45 columns, 203 nonzeros
Variable types: 0 continuous, 45 integer (40 binary)

Root relaxation: objective 0.000000e+00, 15 iterations, 0.00 seconds

    Nodes    |    Current Node    |     Objective Bounds      |     Work
 Expl Unexpl |  Obj  Depth IntInf | Incumbent    BestBd   Gap | It/Node Time

     0     0    0.00000    0    5 5335.00000    0.00000   100%     -    0s
H    0     0                     320.0000000    0.00000   100%     -    0s
     0     0    0.00000    0    6  320.00000    0.00000   100%     -    0s
     0     0    0.00000    0    5  320.00000    0.00000   100%     -    0s
     0     0    0.00000    0    6  320.00000    0.00000   100%     -    0s
     0     0    0.00000    0    5  320.00000    0.00000   100%     -    0s
H    0     0                     239.0000000    0.00000   100%     -    0s
     0     0    0.00000    0    5  239.00000    0.00000   100%     -    0s
*   30     0              29      96.0000000    0.00000   100%   2.7    0s
*   99    32              34      58.0000000    0.00000   100%   2.1    0s
H  506   214                      53.0000000    0.00000   100%   1.9    0s
H30682   442                       1.0000000    1.00000  0.00%   2.1    0s

Cutting planes:
  Cover: 26

Explored 30682 nodes (65348 simplex iterations) in 0.70 seconds
Thread count was 4 (of 4 available processors)

Optimal solution found (tolerance 1.00e-04)
Best objective 1.000000000000e+00, best bound 1.000000000000e+00, gap 0.0%
Optimal objective: 1
```



rounding LP solution
diving at some nodes

Primal Heuristics

search feasible solutions locally around the LP solution

accelerate the search a little
appeal to the practitioner a lot

limits of branch&cut

- all-purpose vs tailored solver
- highly heuristic (branching decisions, cut generation)
- floating-point errors and optimality tolerance (0.01%)
- less effective on integers vs binaries (ex: scheduling)
- MILP approximations for nonlinearities are either large or loose
- NP-hard problems

how to tune modern solvers

play with Gurobi

https://www.gurobi.com/wp-content/uploads/intro_tuning.pdf

(or read the manual)


```
Root relaxation: objective 0.000000e+00, 15 iterations, 0.00 seconds

    Nodes    |    Current Node    |     Objective Bounds      |     Work
 Expl Unexpl |  Obj  Depth IntInf | Incumbent    BestBd   Gap | It/Node Time

     0     0    0.00000    0    5 5335.00000    0.00000   100%     -    0s
H    0     0                     320.0000000    0.00000   100%     -    0s
     0     0    0.00000    0    6  320.00000    0.00000   100%     -    0s
     0     0    0.00000    0    5  320.00000    0.00000   100%     -    0s
     0     0    0.00000    0    6  320.00000    0.00000   100%     -    0s
     0     0    0.00000    0    5  320.00000    0.00000   100%     -    0s
H    0     0                     239.0000000    0.00000   100%     -    0s
     0     0    0.00000    0    5  239.00000    0.00000   100%     -    0s
*   36     0              29      96.0000000    0.00000   100%   2.7    0s
    99    32              34      58.0000000    0.00000   100%   2.1    0s
H  506   214                      53.0000000    0.00000   100%   1.9    0s
H30682   442                       1.0000000    1.00000  0.00%   2.1    0s
```

use as a heuristic

set a time limit or loose gap

```
MIPFocus = 1
ImproveStartGap = 0.1
TimeLimit = 600
MIPGap = 1e-1
```

```
Root relaxation: objective 0.000000e+00, 15 iterations, 0.00 seconds

    Nodes    |    Current Node    |     Objective Bounds      |     Work
 Expl Unexpl |  Obj  Depth IntInf | Incumbent    BestBd   Gap | It/Node Time

     0     0    0.00000    0    5 5335.00000    0.00000   100%     -    0s
H    0     0                     320.0000000    0.00000   100%     -    0s
     0     0    0.00000    0    6  320.00000    0.00000   100%     -    0s
     0     0    0.00000    0    5  320.00000    0.00000   100%     -    0s
     0     0    0.00000    0    6  320.00000    0.00000   100%     -    0s
     0     0    0.00000    0    5  320.00000    0.00000   100%     -    0s
H    0     0                     239.0000000    0.00000   100%     -    0s
     0     0    0.00000    0    5  239.00000    0.00000   100%     -    0s
*   36     0              29      96.0000000    0.00000   100%   2.7    0s
*   99    32              34      58.0000000    0.00000   100%   2.1    0s
H  506   214                      53.0000000    0.00000   100%   1.9    0s
H30682   442                       1.0000000    1.00000  0.00%   2.1    0s
```

change the LP solver

if nbIter(node) ≥ nbIter(root)/2

NodeMethod=2


```
Root relaxation: objective 0.000000e+00, 15 iterations, 0.00 seconds

    Nodes    |    Current Node    |     Objective Bounds      |     Work
 Expl Unexpl |  Obj  Depth IntInf | Incumbent    BestBd   Gap | It/Node Time

     0     0    0.00000    0    5 5335.00000    0.00000   100%     -    0s
H    0     0                     320.0000000    0.00000   100%     -    0s
     0     0    0.00000    0    6  320.00000    0.00000   100%     -    0s
     0     0    0.00000    0    5  320.00000    0.00000   100%     -    0s
     0     0    0.00000    0    6  320.00000    0.00000   100%     -    0s
     0     0    0.00000    0    5  320.00000    0.00000   100%     -    0s
H    0     0                     239.0000000    0.00000   100%     -    0s
     0     0    0.00000    0    5  239.00000    0.00000   100%     -    0s
*   36     0              29      96.0000000    0.00000   100%   2.7    0s
*   99    32              34      58.0000000    0.00000   100%   2.1    0s
H  506   214                      53.0000000    0.00000   100%   1.9    0s
H30682   442                       1.0000000    1.00000  0.00%   2.1    0s
```

init with a feasible solution

if built-in heuristics fail

```
PumpPasses,MinRelNodes,ZeroObjNodes
model.read("initSol.mst")
model.cbSetSolution(vars, newSol)
```



```
Root relaxation: objective 0.000000e+00, 15 iterations, 0.00 seconds

    Nodes    |    Current Node    |     Objective Bounds      |     Work
 Expl Unexpl |  Obj  Depth IntInf | Incumbent    BestBd   Gap | It/Node Time

     0     0    0.00000    0    5 5335.00000    0.00000   100%     -    0s
H    0     0                     320.0000000    0.00000   100%     -    0s
     0     0    0.00000    0    6  320.00000    0.00000   100%     -    0s
     0     0    0.00000    0    5  320.00000    0.00000   100%     -    0s
     0     0    0.00000    0    6  320.00000    0.00000   100%     -    0s
     0     0    0.00000    0    5  320.00000    0.00000   100%     -    0s
H    0     0                     239.0000000    0.00000   100%     -    0s
     0     0    0.00000    0    5  239.00000    0.00000   100%     -    0s
*   36     0              29      96.0000000    0.00000   100%   2.7    0s
*   99    32              34      58.0000000    0.00000   100%   2.1    0s
H  506   214                      53.0000000    0.00000   100%   1.9    0s
H30682   442                       1.0000000    1.00000  0.00%   2.1    0s
```

tighten the model

if the LP bound stagnates

```
Cuts = 3
Presolve = 3
model.cbCut(lhs, sense, rhs)
```


you know your problem better
than your solver does

improve the model

Uncapacitated Facility Location Problem

14 hours

$$\min \sum_{j=1}^{n} c_j x_j + \sum_{j=1}^{n}\sum_{i=1}^{m} d_{ij} y_{ij}$$

$$\text{s.t.} \sum_{j=1}^{n} y_{ij} = 1 \qquad i = 1..m$$

$$\boxed{\sum_{i=1}^{m} y_{ij} \le m x_j \qquad j = 1..n}$$

$$x_j \in \{0,1\} \qquad j = 1..n$$

$$y_{ij} \in \{0,1\} \qquad j = 1..n,\ i = 1..m$$

Input n facility locations, m
pen
serve
y j
ng and
t of

2 seconds

$$\min \sum_{j=1}^{n} c_j x_j + \sum_{j=1}^{n}\sum_{i=1}^{m} d_{ij} y_{ij}$$

$$\text{s.t.} \sum_{j=1}^{n} y_{ij} = 1 \qquad i = 1..m$$

$$y_{ij} \le x_j \qquad j = 1..n,\ i = 1..m$$

$$x_j \in \{0,1\} \qquad j = 1..n$$

$$y_{ij} \in \{0,1\} \qquad j = 1..n,\ i = 1..m$$

m=n=40

Uncapacitated Lot Sizing Problem

$$\min \sum_{t=1}^{n} f_t y_t + \sum_{t=1}^{n} p_t x_t + \sum_{t=1}^{n} h_t s_t$$

$$\text{s.t. } s_{t-1} + x_t = d_t + s_t \qquad t = 1..n$$

$$x_t \le M y_t \qquad t = 1..n$$

$$y_t \in \{0,1\} \qquad t = 1..n$$

$$s_t, x_t \ge 0 \qquad t = 1,\ldots,n$$

$$s_0 = 0$$

ds, fixed
, unit production
age cost h_t,
period t
production and
uction plan to

Uncapacitated Lot Sizing Problem

$$\min \sum_{t=1}^{n} f_t y_t + \sum_{t=1}^{n} p_t \sum_{i=t}^{n} z_{ti} + \sum_{t=1}^{n} h_t \sum_{i=1}^{t}\sum_{j=t+1}^{n} z_{ij}$$

$$\text{s.t. } \sum_{i=1}^{t} z_{it} = d_t \qquad t = 1..n$$

$$0 \le z_{ti} \le d_i y_t \qquad t = 1..n;\ i = t..n$$

$$y_t \in \{0,1\} \qquad t = 1..n.$$

LP=ILP

ds, fixed
, unit production
age cost h_t,
period t
production and
uction plan to

z_{ij} production in period i to satisfy demand of period j

Bin Packing Problem

$$
\begin{array}{lll}
\min \sum_{i=1}^{n} y_i & & \\
\text{s.t.} \sum_{j=1}^{m} w_j x_{ij} \le c y_i & & i = 1..n \\
\sum_{i=1}^{n} x_{ij} = 1 & & j = 1..m \\
x_{ij} \in \{0,1\} & & i = 1..n;\ j = 1..m \\
y_i \in \{0,1\} & & i = 1..n
\end{array}
$$

tainers, m items,
for all containers,
or each item j
cking of all items
m number of

how to manage the exponential number of variables ?

Bin Packing Problem

$$
\begin{array}{lll}
\min \sum_{s \in \mathscr{S}} x_s & & \\
\text{s.t.} \sum_{s \in \mathscr{S}} a_{js} x_s = 1 & & j = 1..n \\
x_s \in \{0,1\} & & s \in \mathscr{S}
\end{array}
$$

Dantzig-Wolfe decomposition

tainers, m items,
for all containers,
or each item j
cking of all items
m number of

$\mathscr{S}$ all the possible arrangements of items in a bin

delayed column generation (for LP)

$min\{c_B x_B + c_N x_N \,|\, A_B x_B + A_N x_N = b\}$ without (c_N, A_N) i.e. $x_N = 0$:

1/ solve the restricted LP with the primal simplex algorithm where the omitted columns N are implicitly non-basic
2/ find $j \in N$ that can profitably enter the basis $\bar{c}_j < 0$, stop if none

same as dual cut generation, cut separation = pricing problem:

$\min cx$		$\max ub$	
$A_i x \ge b_i,$	$\forall i$	$uA_j \le c_j,$	$\forall j$
$x_j \ge 0,$	$\forall j$	$u_i \ge 0,$	$\forall i$

given a basic dual solution u find j such that $\bar{c}_j = c_j - uA_j < 0$

ex: application to (fractional) bin packing

$\mathscr{S} \subseteq 2^m$ all the possible arrangements of items in a bin.

1. let $S \subset \mathscr{S}$ feasible, ie. covering all the items
2. solve the restricted LP, get primal/dual solutions $(\bar{x}, \bar{u})$
   $\min\{\sum_{s \in S} x_s \,|\, \sum_{s \in S} a_{js} x_s = 1\ \forall j,\ x_s \ge 0\ \forall s \in S\}$
3. look for an improving basic direction $s \in \mathscr{S} \backslash S$ with
   $\bar{c}_s = 1 - \sum_j a_{js} \bar{u}_j < 0$, e.g solve the 0/1-knapsack problem:
   $a^* \in \arg\max\{\sum_j a_j \bar{u}_j \,|\, \sum_j w_j a_j \le K, a \in \{0,1\}^m\}$, get $s = \{j : a_j^* = 1\}$
4. if $\sum_j a_j^* \bar{u}_j \le 1$ STOP: $(\bar{x}, 0)$ solves the complete LP over $\mathscr{S}$
5. add s to S then goto 1.

Branch-and-Price (for IP)

- branch-and-bound for ILP with large number of variables where the LP relaxation is solved by column generation
- the branching strategy keeps both the search tree balanced and the LP relaxation structure

  ex bin packing: find a pair of items (i,j) s.t. $0 < \sum_s a_{is}a_{js}x_s < 1$

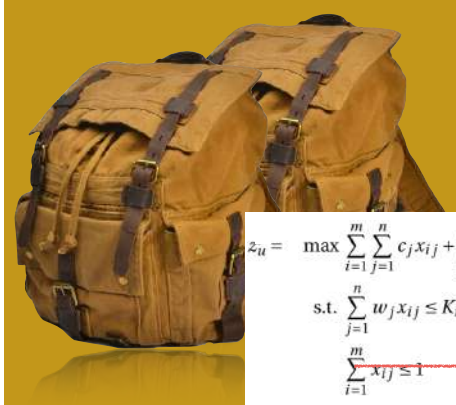
  branch by fixing to 0 either all $x_s \mid \{i,j\} \subseteq s$ or all $x_s \mid \{i,j\} \nsubseteq s$

- the pricing problem is not solved at optimality, except for the convergence proof
- this decomposition method is convenient when additional constraints only appear in the pricing problem

  ex bin packing: conflicts constraint between items $\sum_{j \in C} a_j \leq 1$

Multi 0-1 Knapsack Problem

Input n items, m bins, value c_j and weight w_j for each item j, capacity K_i for each bin i.
Output a maximum value subset of items packed in the bins.



Multi 0-1 Knapsack Problem

$$
\begin{aligned}
z_u = \quad & \max \sum_{i=1}^{m}\sum_{j=1}^{n} c_j x_{ij} + \sum_{j=1}^{n} u_j\Big(1 - \sum_{i=1}^{m} x_{ij}\Big) && u \in \mathbb{R}^n_+ \\
& \text{s.t.} \sum_{j=1}^{n} w_j x_{ij} \leq K_i && i = 1..m \\
& \cancel{\sum_{i=1}^{m} x_{ij} \leq 1} && \cancel{j = 1..n} \\
& x_{ij} \in \{0,1\} && j = 1..n, i = 1..m
\end{aligned}
$$

find the smallest upper bound

lagrangian relaxation

m bins, value
for each item
or each bin i.
m value subset
in the bins.

Lagrangian Relaxation

dualize the complicating or coupling constraints of an ILP:

$$
\begin{aligned}
(P): z = \max & \sum_k c_k x_k \\
& \sum_k D_k x_k \leq e_k \\
& A_k x_k \leq b_k, && \forall k \\
& x_k \in \mathbb{Z}^p \times \mathbb{R}^n, && \forall k
\end{aligned}
$$

$$
\begin{aligned}
(D): w = & \min_{u \geq 0} l(u) \\
& l(u) = ue + \sum_k z_k^u \\
(P_u): z_u^k = & \max c_k x_k - u D_k x_k \\
& A_k x_k \leq b_k \\
& x_k \in \mathbb{Z}^p \times \mathbb{R}^n
\end{aligned}
$$

(D) is the lagrangian dual problem

(P_u) is the lagrangian suproblem with multipliers u

strong duality may not hold if p>0, ie the dual only provides an upper bound $w \geq z$

solving the lagrangian dual

$$
\begin{aligned}
(P): z = \max \sum_k c_k x_k & \\
\sum_k D_k x_k \le e_k & \\
A_k x_k \le b_k, & \quad \forall k \\
x_k \in \mathbb{Z}^p \times \mathbb{R}^n, & \quad \forall k
\end{aligned}
\qquad
\begin{aligned}
(D): w = \min_{u \ge 0} l(u) & \\
l(u) = ue + \sum_k z_k^u & \\
(P_k^u): z_k^u = \max c_k x_k - u D_k x_k & \\
A_k x_k \le b_k & \\
x_k \in \mathbb{Z}^p \times \mathbb{R}^n &
\end{aligned}
$$

- l is a convex piecewise linear function and a subgradient at $u \ge 0$ is $e - \sum D_k x_k^u$ where x_k^u an optimal solution of (P_k^u)
- minimize l with a subgradient, bundle, or cutting-plane method
- solutions x_k^u are almost feasible in (P): try to repair violations heuristically to compute feasible solutions and lower bounds at each iteration k

logic & constraint programming

integer nonlinear programming

graph algorithms

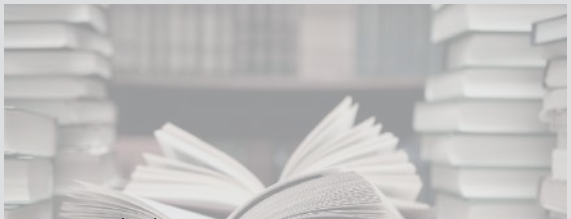
combinatorial optimization beyond MILP

machine learning

dynamic programming

metaheuristics

Matteo Fischetti (2019) Introduction to Mathematical Optimization
Wolsey L. (1998) Integer Programming. Wiley
Bertsimas D., Tsitsiklis J. (1997) Introduction to Linear Optimization. Athena Scientific



Achterberg T., Berthold T., Hendel G. (2011) Rounding and propagation heuristics for MIP. In OR Proceedings 2011, 71-76.
Achterberg T., Bixby R., Gu Z., Rothberg E., Weninger D. (2016) Presolve reductions in MIP. ZIB-Report 16-44.
Bixby R. (2012) A brief history of LP and MIP computation. Documenta Mathematica, 107-121.
Cornuéjols G. (2008) Valid inequalities for MILP. Mathematical Programming, 112(1), 3-44.
Klotz E., Newman A. (2013) Practical guidelines for solving difficult MILP. Surveys in ORMS, 18(1), 18-32.
Linderoth J., Ralphs T. (2005) Noncommercial software for MILP. IP: theory and practice, 3, 253-303.
Linderoth J., Lodi A. (2010) MILP software. Wiley encyclopedia of ORMS.
Linderoth J., Savelsbergh M. (1999) A computational study of search strategies for MIP. INFORMS JoC, 11(2), 173-187.
Lodi A. (2010) MIP computation. In 50 Years of IP 1958-2008, 619-645.
Newman A., Weiss M. (2013) A survey of linear and mixed-integer optimization tutorials. INFORMS ToE, 14(1) 26-38.
Savelsbergh M. (1994) Preprocessing and probing techniques for MIP. ORSA Journal on Computing, 6(4), 445-454.
Vanderbeck F., Wolsey L. (2010) Reformulation and decomposition of IP. In 50 Years of IP 1958-2008, 431-502.